Joint Prefix-State Repair for Barge-In in Cascaded Spoken Dialogue: Integrity and Recovery Cost

[AUTHOR NAME(S) TO BE CONFIRMED][a,*]

[a] *[DEPARTMENT AND INSTITUTION TO BE CONFIRMED], [POSTAL ADDRESS TO BE CONFIRMED], [CITY TO BE CONFIRMED], [POSTCODE TO BE CONFIRMED], [COUNTRY TO BE CONFIRMED]*

Abstract

Barge-in leaves generation, synthesis, and playback at different positions in cascaded spoken-dialogue systems. We implement joint prefix-state repair: a software playback cursor selects a text-to-speech fragment boundary, and the key–value cache, mask, token ledger, positions, and role/end-of-turn state are repaired together. The method distinguishes zero-content playback from complete candidate invalidation and commits structural closure once. Under a frozen Qwen2-7B-Instruct configuration with brain floating-point precision and scaled dot-product attention, 27 production crops matched direct slicing of the same snapshot across 28 layers, and 60 matched recovery steps remained exact. Independent software replay checked 100 stored trajectories at 1118 cursors. A same-policy comparison with executable full rebuilding covered nine fixed cases and 160 paired interruption events across four process sessions. At ordinary fragment boundaries in 512-, 2048-, and 8192-token contexts, mean rebuild-minus-crop first-deliverable costs were 113.621, 465.377, and 1956.100 ms; respective session-cluster 95% confidence intervals were [111.555, 116.132], [464.181, 466.851], and [1940.446, 1968.986] ms. These costs sum synchronized operation intervals, including the first token's cache commit, rather than continuous speech-response latency. First-token choices agreed in all pairs, but eight-token continuations agreed in only 144/160. Downstream automated information-reproduction estimates remained inconclusive. The contribution is a checkable state-repair implementation and

*Corresponding author.

Email address: `[CORRESPONDING AUTHOR EMAIL TO BE CONFIRMED]` ([AUTHOR NAME(S) TO BE CONFIRMED])

bounded recovery-cost evidence, not cross-topology numerical equivalence, acoustic alignment, or demonstrated user benefit.

Keywords: cascaded spoken dialogue, barge-in, playback cursor, prefix-state repair, KV cache, dialogue history

1. Introduction

Barge-in exposes a state-consistency problem that arises in interactive speech systems. When a user begins speaking while an assistant response is playing, language-model generation, text-to-speech (TTS) synthesis, and software playback may have advanced to different positions. The language model may already have generated a suffix that the synthesizer has only partly converted and the player has not consumed. Committing the complete generated response can make the next turn depend on information outside the observed playback path. Discarding the whole assistant turn instead loses both the retained prefix and its computation. Stopping audio is therefore only one part of barge-in handling: a cascaded system must also decide which assistant prefix remains committed and make every representation of the conversation agree with that decision.

This problem requires a precise account of progress. We distinguish *software-consumed samples*, *device-presented samples*, and *acoustically heard content*. The first quantity is reported by playback software. The second additionally depends on application, operating-system, driver, and device buffers, while the third is further affected by transduction, propagation, and the listener's environment. This work observes only a software-consumed-sample cursor. Accordingly, "playback-conditioned" denotes state operations driven by this software cursor, not token-level knowledge of what reached the device or what a listener heard.

Cascaded automatic speech recognition (ASR)–large language model (LLM)–TTS systems remain attractive because their components can be selected, instrumented, and optimized separately. Recent work has addressed semantic triggering, incremental reasoning, interruption control, and streaming synthesis (Zou et al., 2026; Mai, 2026; Chen et al., 2025; Du et al., 2024). Predictive ASR can also use partial recognition to prepare downstream results before the input is final (Schwarz et al., 2023). Within *Speech Communication*, adjacent studies have examined low-latency integration for telephone-conversation processing (Morrone et al., 2024), the effect of telecommunications latency on speaker transitions (Edwards, 2025), and conversation-history prompting for speech-LLM ASR (Zheng et al., 2026). Faster or earlier computation, however, does not determine which generated content should survive a barge-in.

Playback-conditioned history truncation is established engineering practice. The OpenAI Realtime API accepts an `audio_end_ms` boundary and removes unplayed audio together with the associated transcript (OpenAI, n.d.). Azure Voice Live provides `auto_truncate`, which updates the previous response and session context following an interruption and documents a real-time-playback assumption in its estimate (Microsoft, 2026). LiveKit exposes interruption semantics intended to align stored transcript history with spoken output (LiveKit, n.d.). We therefore do not claim the general principle that history should reflect played output. Nor do we treat cache shortening as novel, since Transformers already provides a `DynamicCache.crop` operation (Hugging Face, 2025b,a).

The research object is narrower and cross-layer: an *external-progress-conditioned joint prefix-state repair contract.* A software cursor is resolved through TTS-fragment metadata to a legal assistant commit boundary. That boundary is then applied consistently to the transformer key–value (KV) cache, attention mask, complete token ledger, assistant-content span, position progression, and chat role/end-of-turn (EOT) state. These components cannot be repaired independently. A cache of the intended length may still encode an invalid conversation if its mask, ledger, positions, or role boundary refers to a different prefix. Conversely, a transcript edit does not ensure that the retained model state represents that transcript.

The contract has four layers. First, *boundary resolution* maps the software cursor through a TTS fragment to an assistant token span. Second, *joint-state representation* treats the cache, mask, ledgers, content span, positions, and role/EOT state as one coupled object. Third, *invariant-preserving transitions* define cropping and role reopening without duplicating or misclassifying structural EOT. Fourth, *falsifiable validation* compares the production crop with direct layer-wise slicing of the same pre-crop snapshot, applies wrong-length negative controls, and checks matched recovery from identical retained states.

The evaluation follows three linked questions: does the implementation preserve the specified state, what operational cost does reuse avoid relative to rebuilding the same retained history, and what remains uncertain at the software boundary and in downstream responses? These questions support one method contribution rather than independent claims for triggering, cache management, and history rewriting. The same-policy recovery comparison extends an isolated repair microbenchmark to an executable next-turn state and the first generator-deliverable token.

The sole core mechanism contribution is the implemented path

$$\begin{aligned} &\text{software cursor} \to \text{TTS fragment} \to \text{assistant token span} \\ &\qquad \to \text{KV crop} \to \text{role/EOT recovery.} \end{aligned} \tag{1}$$

Under a frozen Qwen2-7B-Instruct, Transformers, BF16, and scaled dot-product attention (SDPA) configuration (Yang et al., 2024), the accepted exact-only evaluation covered 24 cases, 27 crop events, and 60 recovery steps. The retained production state was bitwise exact across 28 K/V layers against direct layer-wise slicing of the same snapshot, all wrong-length controls were detected, and matched recovery arms remained stepwise exact within the accepted run. These results support direct crop integrity and within-run matched-arm recovery exactness only. Earlier clean-reprefill protocols remain rejected; the accepted protocol does not establish clean-reprefill or continuation equivalence, or correctness across models, backends, or hardware.

Two complementary evaluations connect that integrity result to use. Independent replay checks stored software-cursor/fragment boundaries, while a four-session comparison measures crop repair against executable full rebuilding under the same retention policy. Across nine fixed cases, reuse reduces measured recovery and first-deliverable operation costs; the comparison does not cover optimized serving strategies that already reuse stable prefixes. The fixed-trajectory downstream probe remains inconclusive and is retained here because implementation integrity alone cannot establish dialogue benefit.

The accompanying Supplementary Material supplies complete candidate-generation and implementation-path results (C1/C-E1/C-E2), exploratory history naturalization (A2), downstream sensitivities, and artifact pointers. Both favorable oracle contrasts and unfavorable or confounded comparisons are preserved there; none supplies the main contribution claim. The article is an incremental systems study of inspectable repair and bounded operational cost, not a claim of a new playback policy or major algorithmic novelty.

2. Related work

2.1. Playback-conditioned transcript and history truncation

The closest precedents at the interaction-policy level revise conversation state according to playback progress. OpenAI removes the unplayed audio and transcript suffix at a client-supplied audio boundary (OpenAI, n.d.). Azure similarly updates the response and session context after an interruption (Microsoft, 2026), while LiveKit exposes framework-level semantics for truncating interrupted speech (LiveKit, n.d.). These systems establish that an assistant message need not be retained according to generation completion.

Their public interfaces do not establish their inference internals. The reviewed OpenAI and Azure documentation does not disclose whether the services use a cascaded architecture, how assistant token spans are represented, or whether KV state is cropped in place. LiveKit exposes message and playback behavior but not joint recovery of transformer K/V tensors, masks, token ledgers, positions, and role/EOT state. Our distinction is therefore one of inspectable mechanism and evidence, not an assertion that these systems lack an undisclosed implementation. Their playback quantities are also software-level values or estimates, not direct measurements of device presentation or acoustic reception.

2.2. Streaming dialogue, anticipatory computation, and interruption control

Predictive ASR uses partial recognition to forecast completed input and prefetch a response that can be used if the final recognition confirms the prediction (Schwarz et al., 2023). LTS-VoiceAgent combines semantic triggering with incremental reasoning (Zou et al., 2026), and RelayS2S uses response-level candidate prefixes with verification and continuation (Mai, 2026). These studies establish compute-before-commit as a relevant design pattern for speech interaction. Here its relevance is the need to remove an unaccepted candidate completely when new input arrives. Trigger quality and anticipatory timing are distinct from the repair operation; their separate characterization is reported in Supplementary Material S1.

Interruption-control research primarily determines when output should stop. FireRedChat combines streaming or personalized voice-activity detection with interruption control in cascaded and semi-cascaded configurations (Chen et al., 2025). In contrast, joint prefix-state repair starts after an interruption decision and determines what part of the assistant turn remains committed. Detection and repair are complementary, but successful detection does not imply a valid next-turn cache and dialogue history.

Full-duplex speech models provide another reference. Moshi models concurrent speech and text streams to support overlapping interaction (Défossez et al., 2024). Tighter stream coordination can avoid some boundaries introduced by independently deployed ASR, LLM, and TTS components, but model-stream position, device presentation, and acoustic reception remain distinct in the presence of queues and buffers. Comparisons must therefore name the observed boundary instead of inferring delivery from an architectural label.

Related studies connect streaming speech processing, conversational timing, and contextual inference. Morrone et al. integrated speech separation and voice-activity detection for low-latency diarization (Morrone et al., 2024); Edwards measured how telecommunications delay changes speaker-transition timing (Edwards, 2025); and Zheng et al. combined conversation history and hotwords for contextual prompting in conversational ASR (Zheng et al., 2026). These works address speech processing, latency, and conversational context, respectively. None of them addresses playback-conditioned transformer-state rollback.

2.3. KV-cache management and cross-turn state

Autoregressive transformers retain key and value tensors for prior tokens to avoid recomputing the full prefix. Transformers exposes this state through cache abstractions, including `DynamicCache.crop` (Hugging Face, 2025b,a). The primitive makes in-place truncation possible but neither selects the conversational boundary nor repairs non-cache state.

Serving research addresses adjacent objectives. PagedAttention organizes KV memory into non-contiguous blocks with demand allocation and copy-on-write (Kwon et al., 2023). SGLang's RadixAttention manages reusable token prefixes in a radix tree (Zheng et al., 2024). These methods improve memory use, throughput, or shared-prefix reuse rather than select an interrupted assistant prefix from software playback progress. IntentKV prunes cross-turn state according to text-agent intent (Li et al., 2026), and Speculative Interaction Agents coordinate provisional results around asynchronous tool calls (Hooper et al., 2026). Their control signals are text intent or tool state, not TTS-fragment ownership and software-consumed samples.

The technical distinction of the present work is consequently not a new cache representation or crop operator. It couples an external software progress signal to a legal conversational prefix, then repairs all representations of that prefix in one transition. Its validation is equally specific: direct slicing of the same pre-crop snapshot checks whether the requested K/V prefix was retained; a wrong-length control checks whether the gate can reject a boundary error; matched recovery checks whether equal retained states remain equal under identical subsequent token chunks and operations. These gates do not establish equivalence to independently recomputed prefixes, semantic correctness of subsequent responses, or portability across inference engines.

A targeted public-source scan was frozen on 3 September 2026. It covered cross-publisher indexes, arXiv, ACL Anthology, ISCA Archive, ACM Digital Library, IEEE Xplore, NeurIPS proceedings, DOI/Crossref metadata, and first-party product and repository sources. Queries combined cascaded speech dialogue, barge-in, anticipatory response generation, cache rollback, and playback-aware history. Within the reported and accessible sources, no publicly inspectable cascaded implementation was identified that disclosed the complete cursor–fragment–token–cache–role path together with reproducible direct-integrity and recovery evidence. Some sources were inaccessible because of automated-access restrictions, and the search was neither a systematic review nor a patent search. This is a bounded non-identification result, not a global-first claim.

3. System model and problem formulation

3.1. Cascaded streaming system

We consider a cascaded spoken-dialogue system

$$\mathcal{S} = \langle \mathrm{ASR}, \mathrm{LLM}, \mathrm{CHK}, \mathrm{TTS}, \mathrm{PLY} \rangle, \tag{2}$$

whose components communicate incrementally. Streaming ASR converts an utterance into stable text segments $U = \langle u_1, u_2, \ldots \rangle$. Downstream processing consumes only final segments rather than mutable partial hypotheses. The LLM incrementally prefills these segments and generates assistant content $Y = \langle y_0, \ldots, y_{G-1} \rangle$, where G is the current content endpoint.

A chunker partitions decoded assistant output into complete text fragments $F = \langle f_1, \ldots, f_m \rangle$ for TTS. Each fragment owns a contiguous, half-open interval on the assistant-content token axis:

$$\begin{aligned} f_j &\mapsto [\mathrm{ts}(f_j), \mathrm{te}(f_j)), \\ \mathrm{ts}(f_1) = 0, &\qquad \mathrm{ts}(f_{j+1}) = \mathrm{te}(f_j). \end{aligned} \tag{3}$$

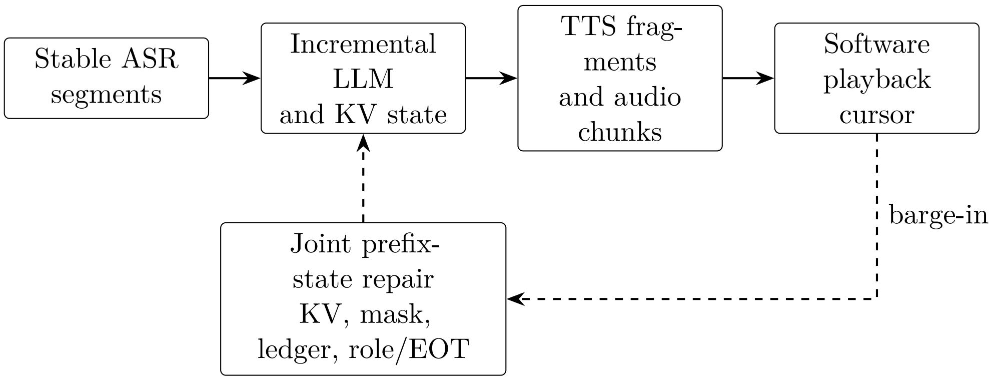

Figure 1: Cascaded architecture and interruption-time state repair. The cursor is a software-level signal; the figure does not imply a device or acoustic boundary.

A fragment is the atomic unit of retention because the system has no word-, phoneme-, or token-level alignment between synthesized audio and text. Streaming TTS transforms each fragment into one or more audio chunks. Once registered, fragment f_j occupies the cumulative sample interval $[\mathrm{ss}(f_j), \mathrm{se}(f_j))$ at sampling rate r. The player reports a cursor $p \in \mathbb{N}$ with counting semantics: $[0, p)$ has been consumed in software.

Figure 1 gives the processing path. ASR segments u_i and TTS fragments f_j are distinct: the former drive input-side prefill, while the latter associate assistant tokens with synthesized samples.

3.2. Software retention boundary

At time t, let $G(t)$ denote the generated-content endpoint and $S(t)$ the largest token endpoint registered on the TTS/software timeline. Because $p(t)$ lies on a sample axis, it cannot be compared directly with $G(t)$ or $S(t)$. If p lies within fragment f_k under the counting convention

$$\mathrm{ss}(f_k) < p \leq \mathrm{se}(f_k), \tag{4}$$

the fragment-level retention boundary is

$$\widehat{H}(p) = \mathrm{te}(f_k). \tag{5}$$

When the cursor falls inside f_k, the boundary is deliberately rounded upward to the complete fragment endpoint. This policy retains the hit fragment's unconsumed tail as well as its consumed prefix; it does not recover an exact

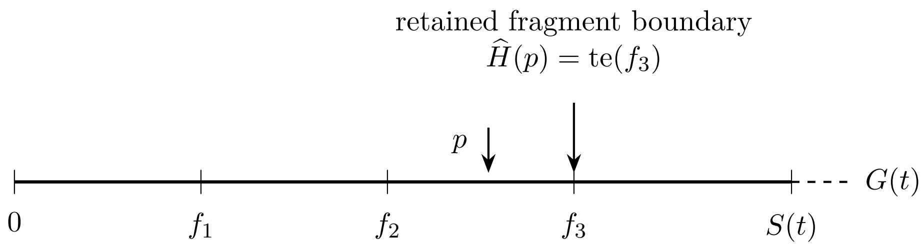

Figure 2: Generated, registered, and software-consumed progress after conversion to the token domain. A cursor inside f_3 selects its complete fragment endpoint.

transcript of consumed audio. Excluding the hit fragment instead would remove material already consumed in software. Whole-fragment retention is therefore an explicit granularity tradeoff, not an empirically established optimal policy. A separate partial flag records the intra-fragment hit without correcting that approximation. For $p = 0$, $\widehat{H}(0) = 0$. A cursor beyond registered audio is clamped to the last fragment with an audio record.

Under the producer-order assumption that playback consumes only registered fragments and fragments enter the timeline in generation order,

$$\widehat{H}(p(t)) \leq S(t) \leq G(t). \tag{6}$$

This is an application-timeline invariant, not a model of acoustic propagation. The reverse query is

$$\Phi : p \mapsto f_k \mapsto [\mathrm{ts}(f_k), \mathrm{te}(f_k)) \mapsto \widehat{H}(p). \tag{7}$$

It is an association across samples, chunks, fragments, and token spans rather than an invertible alignment. Figure 2 illustrates the three progress quantities after they have been mapped to a common token domain.

For downstream analysis only, a partially consumed fragment tail is approximated from the character ratio

$$\alpha(p) = \frac{p - \mathrm{ss}(f_k)}{\mathrm{se}(f_k) - \mathrm{ss}(f_k)}. \tag{8}$$

The raw split $\mathrm{round}(\alpha(p)L_k)$ for fragment length L_k is moved backward to the nearest whitespace boundary. This proxy is not used as a crop point and is neither token-linear nor an acoustic alignment.

3.3. Persistent joint prefix state

Interruption repair is not an operation on a KV object alone. We define the persistent state as

$$\mathcal{Z} = \langle \mathcal{K}, M, I, A, \varphi, e, a_0, a_1 \rangle, \tag{9}$$

where $\mathcal{K}$ is the dynamic KV cache, M is the attention mask, I is the complete token-ID ledger, A is the current assistant-content ledger, φ is the role phase, e is the generation end reason, and $[a_0, a_1)$ is the absolute assistant-content span. Every stable state satisfies

$$|I| = |M| = \mathrm{seq}(\mathcal{K}), \qquad A = I[a_0 : a_1]. \tag{10}$$

Positions for subsequent prefill are derived from the actual cache length after each operation.

The role phase belongs to three states: `USER_OPEN`, `ASSISTANT_OPEN`, and `ASSISTANT_EOT_PENDING`. The end reason explicitly distinguishes none, model-selected EOT, maximum-token termination, consumer stop, and cropping. A selected structural EOT moves the role to `ASSISTANT_EOT_PENDING` and records EOS, but is not forwarded into $\mathcal{K}$, appended to A, or admitted to TTS. A later role-reopening operation is the only close commit and appends exactly one template-derived assistant-close sequence followed by user-open tokens.

For playback interruption, the relative boundary is converted to the absolute endpoint

$$N = a_0 + \widehat{H}(p). \tag{11}$$

The repaired state must shorten K/V, mask, global ledger, assistant ledger, and content span to one semantic boundary; remove all access to discarded content; continue positions from the retained prefix; and assign role/end state consistent with the retained token sequence.

Two zero-content cases are distinct. A playback interruption with $\widehat{H}(p) = 0$ crops to a_0, preserving the assistant header and an open assistant role before normal closure. Full invalidation of an unaccepted candidate instead returns to the pre-speculation `USER_OPEN` endpoint and removes both the assistant header and candidate content. Conflating the two cases either manufactures an empty assistant turn or erases a turn that already entered the output path.

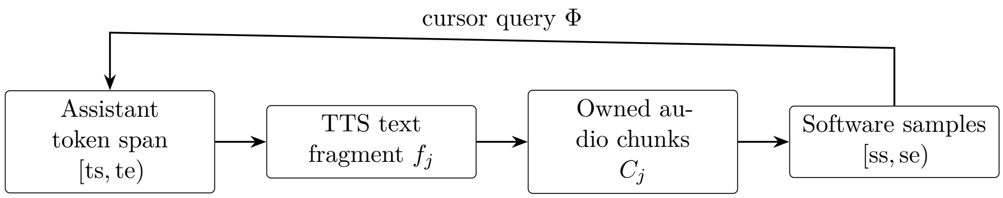

Figure 3: Fragment association record. A TTS fragment links one assistant-token span to its ordered audio chunks and cumulative software-sample interval.

4. Playback-conditioned joint prefix-state repair

4.1. Fragment association and producer contract

Each timeline record contains

$$\langle f_j, [\mathrm{ts}, \mathrm{te}), C_j, [\mathrm{ss}, \mathrm{se}), s_j \rangle, \tag{12}$$

where C_j is the ordered list of audio chunks and s_j is the fragment lifecycle state. Generation registers a frozen token span, TTS attaches chunks and advances the cumulative sample axis, and the player advances p. On interruption, Φ locates the hit fragment and returns its token endpoint (Figure 3).

The reverse query is valid only if write-side constraints hold. Fragment spans must be non-empty, monotonic, and contiguous. A fragment accepts audio only after its span is frozen, and no terminal fragment accepts further chunks. Every chunk identifier is globally unique. Sample intervals must be non-empty, ordered, and contiguous, while the software cursor is monotonic and cannot advance beyond registered samples. Out-of-order fragments, duplicate or misassigned chunks, gaps, overlaps, post-closure appends, and cursor regression are rejected rather than silently reordered.

The chunker sees decoded text rather than token identifiers. To associate a fragment with a token endpoint, the method accumulates non-whitespace characters contributed by each decoded token and locates the fragment's non-whitespace length in that cumulative sequence. Whitespace-only fragments are skipped, endpoints are clamped to the observed content length, and the concatenated non-whitespace fragment sequence must equal the original decoded sequence. The procedure establishes a deterministic software boundary without claiming acoustic alignment.

4.2. Complete invalidation before output commitment

An unaccepted response candidate has a different rollback boundary from a response that entered playback. Before opening the candidate's assistant

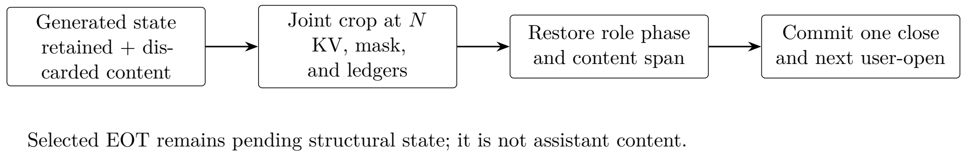

Figure 4: Joint crop and role recovery. A unique close/reopen transition commits structural EOT without duplicating it in assistant content.

role, the system saves the `USER_OPEN` endpoint. If another stable input segment arrives, repair returns to this endpoint, removing both candidate content and its assistant header. Immediately after cropping, `CROPPED` remains observable; successful prefill of new user text resets the current end reason. A subsequent response can open a fresh assistant role. This transition is part of the joint repair method regardless of how the candidate was triggered. The threshold policy and synchronous-oracle timing study are reported separately in Supplementary Material S1.

4.3. Joint crop and role recovery

Given a legal endpoint N, cropping shortens every K/V layer on its sequence dimension and truncates M, I, A, and $[a_0, a_1)$ to the same prefix. It assigns a role phase consistent with the retained structure and marks the current end reason as `CROPPED`. An endpoint inside structural role or EOT tokens is rejected.

For playback interruption, the hit fragment and preceding fragments remain in A, while later content disappears from all cache and ledger representations. When no assistant content is retained, the assistant header remains and the role is still open. The role-reopening operation then appends the template-derived assistant-close and user-open sequence once. If that sequence contains q tokens, the state after reopening satisfies

$$|I| = |M| = \mathrm{seq}(\mathcal{K}) = N + q, \qquad |A| = \widehat{H}(p). \tag{13}$$

The structural tokens enter $\mathcal{K}$, M, and I but not A; the next user text begins at position $N + q$.

Full candidate invalidation instead crops to the saved user-open endpoint before the assistant header and inserts no assistant close. The `CROPPED` state persists until the next user segment is appended. Figure 4 summarizes the playback case and the unique structural close.

Editing text without removing its K/V representation leaves the model conditioned on discarded content. Shortening K/V without truncating the mask and ledgers invalidates lengths and positions. Joint repair makes the same retained prefix inspectable in token space and model state. Here, joint consistency is specified at completed operation boundaries; it is not a claim of transactional atomicity for concurrent readers.

4.4. Exact validation

Direct crop integrity is evaluated by comparing three states at every crop event: the prefix extracted from the pre-crop production state, the state after one production crop, and a slicing oracle cloned from the same pre-crop snapshot. The oracle slices every K/V tensor directly to the derived length together with the mask and global token ledger; it does not call the production crop interface. It is nevertheless a same-snapshot oracle, not an external implementation or clean re-prefill reference. It tests preservation of a specified prefix, not whether the pre-crop state was semantically correct or whether a live audio cursor was mapped to the appropriate text. Those boundary assumptions require separate validation.

The frozen grid contains 24 ordered cases covering 512-, 2048-, and 8192-token contexts; zero-content retention; exact and mid-fragment boundaries; reply-tail removal; pending EOT; full invalidation; a later user turn; and a second crop. The cases produce 27 crop events, including three no-ops. All 308 assistant fixture tokens are appended through the production accumulation path.

For each event, the target length is derived from the case, fragment partition, and role/content boundaries rather than trusted from a recorded keep value. Validation compares per-layer K/V shape, dtype, device, SHA-256, and runtime element equality, together with the mask, token ledger, sequence length, content span, role phase, and end reason. A disposable wrong-length control must fail for each event.

After cropping, the production and oracle arms receive identical token-ID chunks and role/state operations for 60 matched recovery steps. K/V, logits, masks, ledgers, retained-prefix hashes, and independently derived role/end/content states are compared after each step. We call this *within-run matched-arm recovery exactness*. It means that two exactly matched retained states remain equal under the same operations in one accepted run. It does not mean determinism across processes, devices, versions, precisions, or backends, and it does not establish clean-reprefill or continuation equivalence.

Table 1: Evidence roles and limits of inference.

Study	Question	Scope of answer
C2 v3	Is the specified prefix preserved?	Same-snapshot exact crop and matched recovery
B	Does the software cursor select the specified boundary?	Independent replay of stored software records
R	What cost does executable reuse avoid?	Same-policy crop versus full rebuild; fixed fixtures
A1/P1	What are local repair and software-control costs?	Synchronized microbenchmark and prepared-state headless path
E3	Does retention change automated information reproduction?	Inconclusive fixed-trajectory diagnostic

5. Experimental methodology and results

5.1. Evidence and environment

The evaluation separates prefix integrity, operational recovery cost, and downstream effects (Table 1). C2 tests same-snapshot crop integrity and matched recovery. B checks the software boundary independently of the production lookup. R compares executable next-turn recovery under the same retention policy. A1 and P1 provide supporting measurements at different operation boundaries; E3 is a downstream diagnostic. Absolute times from different campaigns are neither pooled nor subtracted.

The principal model was Qwen2-7B-Instruct (Yang et al., 2024). The R runtime used Python 3.10.18, PyTorch 2.8.0+cu128, Transformers 4.57.1, CUDA 12.8, cuDNN 9.10.2, BF16, and SDPA, with exactly one exposed RTX 3090 (24 GB) per process session. It used the historical model artifact also recorded by C2, not Qwen2.5. Model loading was strictly offline. Four independently initialized processes reloaded the model; repetitions within them are not independent sessions.

The downstream data were derived from MultiWOZ 2.1 (Eric et al., 2020). Mistral-7B-Instruct-v0.3 supplied automated E3 judgments (Jiang et al., 2023; Mistral AI, 2024). A six-sentence CosyVoice2-0.5B profile parameterized deterministic Mock-TTS durations (Du et al., 2024); these are not live synthesis measurements. The surrounding implementation reuses Whisper-based ASR (Radford et al., 2023) and stream2sentence version 1.0.0 (Beigel, 2026), but the evaluated records do not constitute a live speech loop.

Table 2: C2 v3 exact-gate coverage.

Check	Result
Cases / crop events	24/24; 27/27
Production assistant append	308 tokens
K/V layers compared	28
Recovery steps	60/60
Wrong-length controls detected	27/27
No-op crops	3/3

Supplementary Material S1–S4 contains the separate candidate/history studies and complete artifact mapping.

5.2. C2 exact correctness and B software boundaries

C2 v1 and v2 retain their rejected verdicts. V1 failed its frozen clean-reprefill numerical criteria; v2 passed structural checks but only 42/45 numerical comparisons. Its control did not match the production forward topology, preventing attribution of numerical differences to cropping. C2 v3 instead posed the narrower same-snapshot slicing and matched-recovery questions. Rejected protocols, records, and their diagnostic limitations remain available in Supplementary Material S4.

At all 27 crop events, the retained pre-crop K/V prefix, production post-crop K/V, and slicing-oracle K/V agreed across 28 layers in shape, dtype, device, hash, and runtime `torch.equal`. The independently derived keep length, mask, global ledger, sequence length, and K/V length were exact. Across 60 recovery steps, matched arms remained exact in K/V, logits, mask, ledger, retained-prefix hash, and role/end/content state. All wrong-length controls were detected. This is direct crop integrity and within-run matched-arm recovery exactness, not clean-reprefill equivalence.

B independently reconstructed expected fragment selection from ordered sample-end intervals and replayed the production timeline on 100 saved E3 trajectories. It checked 800 stored condition records and 1118 distinct trajectory–cursor combinations, covering zero, fragment starts, the next sample, midpoints, ends, total length, and an overrun query. Token/sample continuity, retained/history endpoints, hit identity, partial flags, and lifecycle states passed. It also independently derived the keep lengths and closure ledgers for 27 C2 events, including both zero-content cases and second crops. B thus connects saved software geometry to the declared retention policy without reusing the production lookup as its reference. It cannot validate real TTS alignment, device presentation, or listening truth.

5.3. R: executable same-policy next-turn recovery

Design and comparator.. R used nine frozen, non-Cartesian cases: ordinary fragment boundaries at 512, 2048, and 8192 initial-context tokens; short-context zero playback, mid-fragment upward retention, reply-tail no-op, pending EOT, and complete candidate invalidation; and a 2048-token case with a second interruption. Nine cases produced ten case–event cells. Each of four sessions included one excluded warm-up pair per case and four measured pairs, yielding 144 case pairs, 160 event pairs, and 320 arm–event records. Arm order was balanced by session, case, and repetition. EOT states were controlled fixtures driven through the production generator, not observations of natural termination despite historical case-name suffixes.

Each arm independently prepared the same pre-interruption token history outside the timed intervals. The crop arm used the production crop and, when required, role reopening. The full-rebuild arm forwarded the identical retained token IDs and legal close/user-open sequence in one batch, keeping the resulting cache and logits in an executable state. Both then appended the same user suffix and assistant header and ran greedy decoding for at most eight token selections. Thus R compares the same retention policy and serialized next-turn history, not different history strategies. Full rebuilding is a meaningful recomputation reference but not an optimized stable-prefix serving baseline.

For the second interruption, the first diagnostic continuation was removed outside the measurement window and both arms were advanced through the same frozen second-assistant fixture, while preserving their repaired first-turn histories. This reconciliation prevents numerical greedy divergence from changing the prefixes being compared. It tests a second controlled repair, not unrestricted long-conversation recovery.

Endpoints and uncertainty.. All operation intervals used `perf_counter_ns` with CUDA synchronization at their boundaries. Recovery ends at `USER_OPEN`; suffix/header readiness ends after the next assistant role is opened. The separate decode interval ends when the first `next(generator)` returns and device synchronization completes, including the token's actual KV commit and yield rather than only its selection callback. With T_{suffix}

measured after recovery,

$$T_{\text{deliver}} = T_{\text{recover}} + T_{\text{suffix}} + T_{\text{decode,first}}. \tag{14}$$

This is a sum of synchronized intervals, not a continuous wall-clock span. Model loading, initial history and fixture preparation, CPU state/logit audits between intervals, and the remaining diagnostic continuation are excluded. ASR, TTS, and playback are absent. A first-selection EOT would yield no deliverable and a null cost rather than a fabricated zero; no such missing deliveries occurred.

For each fixed case/event, the estimand is the mean paired difference $T_{\text{rebuild}} - T_{\text{crop}}$. We report 95% percentile intervals from 10,000 session-cluster bootstrap draws (seed 20260906). Repeats and second events remain within their process cluster. With only four sessions these intervals are coarse descriptions of process variation conditional on fixed fixtures, not uncertainty over 160 independent dialogues or over deployment workloads.

Results.. All 320 records passed the frozen structural/finite-value gates and both arms delivered a first token in all 160 pairs. Table 3 reports every case/event rather than selecting only ordinary boundaries. At the three ordinary boundaries, mean first-deliverable savings were 113.621, 465.377, and 1956.100 ms, respectively. At 8192 tokens the recovery-only difference was 1951.675 ms [1937.285, 1963.226], showing that most avoided work was in reconstructing the retained context, not in the first decoding operation. Positive costs here establish the observed operational advantage over full rebuilding within this fixed design; no outcome-sign acceptance gate or broader workload claim was used.

At matched next-turn-ready states, token IDs, masks, sequence/KV lengths, role/content spans, and closure ledgers satisfied the same structural plan. All 320 saved next-token logit arrays were finite float32 arrays of shape $[1, 152064]$. The maximum cross-arm absolute logit difference was 0.59375. Top-1 choices agreed in 160/160 pairs, but bounded eight-token continuations agreed in only 144/160. All 16 differences occurred at case 16, event index 1 (the second interruption), across all sessions and repeats. Logit and output differences are retained as diagnostics: R requires finite values, not cross-topology BF16 equality. These observations support executable same-policy recovery and measured cost savings, not numerical equivalence or broad functionality without loss.

Table 3: R paired rebuild-minus-crop first-deliverable operation cost (ms). All ten fixed case–event cells are shown; each contains four pairs in each of four process sessions. Intervals are 95% session-cluster percentile intervals, not independent-event intervals. Initial context length does not include later fixture/suffix growth.

Case/event	Boundary (initial tokens)	Mean	95% CI
01/0	Zero playback (512)	113.211	[110.305, 116.116]
02/0	Fragment boundary (512)	113.621	[111.555, 116.132]
03/0	Mid-fragment (512)	114.020	[111.044, 117.452]
04/0	Reply-tail no-op (512)	114.138	[112.147, 115.756]
05/0	Pending EOT (512)	114.629	[112.389, 116.868]
06/0	Full invalidation (512)	128.643	[121.606, 133.348]
10/0	Fragment boundary (2048)	465.377	[464.181, 466.851]
16/0	Repeated interruption (2048)	464.295	[462.504, 466.017]
16/1	Repeated interruption (2048)	467.470	[465.568, 469.915]
18/0	Fragment boundary (8192)	1956.100	[1940.446, 1968.986]

Table 4: A1 synchronized repair microbenchmark; 50 repetitions per length.

Context length	256	512	1024	2048	4096	8192
Joint median (ms)	31.616	31.852	31.054	31.519	36.903	48.315
Joint IQR (ms)	2.356	2.162	3.099	1.197	0.635	0.928
Re-prefill/joint	2.254×	4.124×	7.707×	15.020×	25.453×	40.620×

5.4. Supporting repair microbenchmark and software path

A1 compared synchronized joint crop-plus-role repair with complete re-prefill over 256–8192-token contexts. Each length used five warm-ups and 50 recorded repetitions, fixed operation order, and removal of a fixed 32-token suffix. Table 4 and Figure 5 show joint-repair medians of 31.054–48.315 ms. The median re-prefill-to-repair ratio increased from 2.254× at 256 tokens to 40.620× at 8192 tokens. Unlike R, this local microbenchmark does not extend its comparator to a measured next-turn generator endpoint.

P1 crossed three context lengths (512, 2048, 8192) with three cursor proportions (0.25, 0.50, 0.75) and 20 repetitions per cell, yielding 180 prepared-state headless records. All requests and acknowledgements reached the intended software cursor. Cell medians were 0.055–0.062 ms for stop acknowledgement, 0.167–0.176 ms for subsequent GPU synchronization, 0.47–0.50 ms for timeline lookup, 2.44–2.53 ms from stop to crop completion, and 78.6–

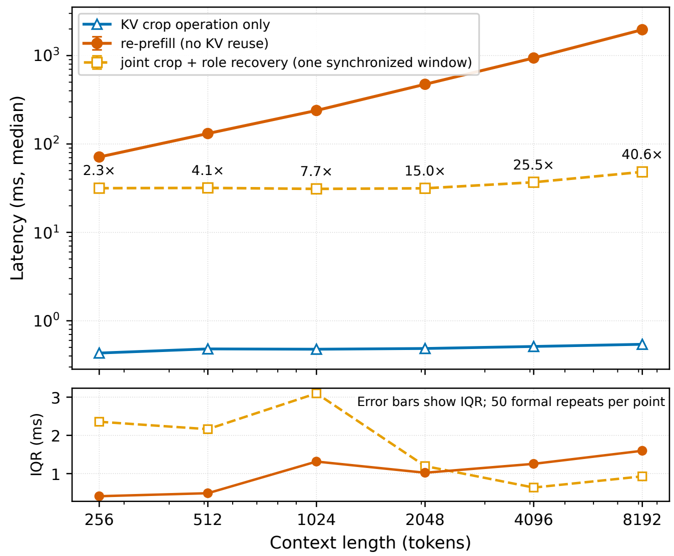

Figure 5: A1 joint repair and re-prefill microbenchmark. Error bars show interquartile ranges across 50 repetitions.

80.8 ms from stop to role recovery. The latter intervals are nested and must not be added. Prepared-state setup was excluded. Zero leaked software samples means only that the software counter did not advance after acknowledgement; it says nothing about sound-card or acoustic stopping. P1's earlier timing-contaminated version remains excluded (Supplementary Material S4).

5.5. Downstream fixed-trajectory diagnostic

E3 contained 100 dialogues, 400 paired dialogue–injection-position units, 800 condition records, and 1600 judge records. Each assistant trajectory was generated once; software-cursor and generation-retention conditions shared that trajectory, its fragment timeline, and four injection positions (0.25, 0.50, 0.75, and a fragment boundary). Fragment and character-ratio proxy targets were separately eligible when their respective target strings were non-empty: 297 fragment labels from 96 dialogues and 380 proxy labels from 100 dialogues.

The primary estimand weights eligible dialogue–position labels equally and measures generation minus software-cursor retention. Intervals use 10,000 paired dialogue-cluster bootstrap repetitions (seed 20260831). The lexical detector matches numbers, selected capitalized terms, and non-

Table 5: E3 label-weighted information-reproduction rates. The effect is generation minus software-cursor retention; pp denotes percentage points.

Target	Detector	Cursor rate	Generation rate	Effect [95% CI] (pp)
Fragment	Lexical	67.00% (199/297)	63.64% (189/297)	−3.37 [−10.49, 3.40]
Fragment	Judge	42.76% (127/297)	40.74% (121/297)	−2.02 [−10.70, 6.13]
Proxy	Lexical	75.26% (286/380)	73.68% (280/380)	−1.58 [−6.08, 2.67]
Proxy	Judge	43.95% (167/380)	41.32% (157/380)	−2.63 [−8.57, 2.90]

stopword content terms. The `specific-reference-v3` Mistral judge receives the target and two delimiter-joined replies without condition identity, and greedily produces a strict YES/NO decision. The intervals describe dialogue-sampling uncertainty conditional on the fixed trajectories, targets, detectors, prompts, and 40-token response cap; they omit detector and model/prompt uncertainty.

All four primary intervals cross zero (Table 5). Dialogue weighting and target-specific exact-key deduplication also leave the direction inconclusive; complete sensitivity results and detector agreement appear in Supplementary Material S3. None establishes superiority, equivalence, non-inferiority, harm, or absence of an effect. In particular, removing complete unretained fragments from the cursor-conditioned history is a construction property, not evidence that subsequent responses reproduce less information or that listeners experience better dialogue.

6. Discussion

6.1. What the combined evidence establishes

The implemented contribution is the coupling between a software progress boundary and all persistent representations of the retained conversational prefix. Transcript-only truncation can leave discarded content accessible through K/V state; tensor-only cropping can leave masks, positions, role structure, or ledgers at an incompatible endpoint. The two zero-content transitions and the pending-EOT lifecycle make these distinctions operational rather than merely descriptive. Joint consistency is checked at completed operation boundaries, not asserted as transactional atomicity for concurrent readers.

The evidence addresses complementary failure modes. C2 checks that the requested prefix survives cropping and matched recovery. B checks software interval selection against an independently constructed reference. R asks whether the repaired state can actually accept the same next-turn input and produce a deliverable token at lower operation cost than full rebuilding. The large long-context difference is consistent with avoiding retained-prefix recomputation. It is not a new asymptotic cache result, but an empirical link between correct conversational serialization and useful recovery work avoided in this implementation.

Structural validity must remain separate from numerical and response identity. R used different forward topologies for reuse and rebuilding: next-token choices agreed in every pair, yet all repetitions of one second-interruption cell produced different short continuations. This is observable output variation, not a tolerance-based equivalence pass. Likewise, the rejected C2 clean-reprefill protocols remain rejected; the exact-only protocol answers a different, identifiable question. Neither the cost advantage nor the structural gates imply that arbitrary subsequent responses are interchangeable.

6.2. Boundary policy and downstream uncertainty

Whole-fragment retention deliberately favors keeping the consumed portion of an interrupted fragment at the cost of also retaining its unconsumed tail. Dropping the fragment would make the opposite error. The partial flag makes this tradeoff visible but does not solve it. B verifies application of the declared policy; it does not show that the policy is perceptually optimal or that simulated sample geometry corresponds to real speech alignment.

E3 is relevant precisely because a correct implementation need not improve dialogue. All four automated point estimates were negative in the generation-minus-cursor direction, but their intervals crossed zero. No substantive equivalence or non-inferiority margin was specified. The lexical and model-based measurements capture different forms of information reproduction and agree imperfectly; exact-key deduplication removes repeated weights, not semantically similar responses. Without blinded human reference labels, E3 cannot establish perceived coherence, trust, or task success.

Its inconclusive result therefore constrains the contribution rather than being replaced by a favorable construction check.

6.3. Validity and transfer conditions

R's four process sessions and fixed cases provide limited replication, not a workload sample or an operational failure-rate estimate. Reconciliation before the second interruption holds the compared prefixes fixed but excludes free-running divergence across many turns. First-deliverable costs are sums of synchronized intervals, excluding preparation and CPU audit pauses. They should not be interpreted as uninterrupted response times or compared directly with P1's nested stop-path intervals. A1 fixes operation order and suffix length, while P1 has only 20 observations per cell; neither supplies a production tail-latency guarantee. The comparator in R reconstructs the full retained prefix and does not represent all serving systems with shared or stable-prefix caches.

The shared external limitation is the observed boundary. Software-consumed samples, device-presented samples, and acoustically heard content differ because of queues, APIs, scheduling, device buffers, propagation, and the listening environment. No reported experiment closes a real streaming-ASR, asynchronous-TTS, and audio-device loop or measures a loopback waveform. The evidence is concentrated on English task-oriented text, a frozen Qwen2-7B artifact, a ChatML-like template, Transformers DynamicCache, BF16/SDPA, and RTX 3090 hardware. Other templates, precisions, backends, languages, or segmentation rules require renewed boundary and state tests.

A transferable implementation therefore needs explicit fragment-to-chunk ownership, an interpretable playback-progress signal, and a backend that retains or reconstructs a legal prefix while updating mask, positions, ledgers, content span, and role/EOT state together. Word-level durations, device clocks, or loopback waveforms could refine the observed boundary, but would change what is being validated. Production claims additionally require concurrency and cancellation tests; interaction-quality claims require direct human evidence. These are extensions of the bounded method, not unmeasured benefits of the current implementation.

7. Conclusion

Joint prefix-state repair turns a software playback boundary into a checkable conversational-state transition. Its implementation distinguishes complete candidate invalidation from zero-content playback, preserves one consistent token/cache prefix, and commits structural closure once. Same-snapshot validation found all 27 crops and 60 matched recovery steps exact, while independent software replay checked 1118 cursor queries over 100 trajectories.

In the executable same-policy comparison, crop repair avoided full-rebuild work through the next-turn first-deliverable endpoint. At ordinary 512-, 2048-, and 8192-token boundaries the mean paired operation-cost differences were 113.621, 465.377, and 1956.100 ms. This is bounded recovery-cost evidence, not continuous speech end-to-end latency or a comparison against every prefix-caching strategy. Short continuations differed in 16/160 pairs despite identical first-token choices, and the downstream automated diagnostic remained inconclusive. The resulting contribution is an inspectable, incrementally useful systems method with explicit correctness and cost boundaries, rather than a new playback principle or a claim of numerical or perceptual equivalence.

CRediT authorship contribution statement

[AUTHOR CONFIRM: list every author and retain only verified CRediT roles. Do not infer roles from repository metadata.]

Funding

[AUTHOR CONFIRM: provide funder names, grant numbers, and funder roles, or state: “This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.”]

Declaration of competing interest

[AUTHOR CONFIRM: declare all relevant financial and non-financial interests for every author, or state: “The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.”]

Data and code availability

The accompanying Supplementary Material S4 maps each reported result to its local artifact, manifest, and analysis. The source package contains both manuscript and supplementary sources; it is not the complete experimental archive. [AUTHOR CONFIRM: provide a deposited dataset/code citation and accessible link, with release identifier and DOI if available, or a factual, justified explanation of why sharing is not possible under the journal's data Option C.] The E3 processed input was derived deterministically from MultiWOZ 2.1; the raw MultiWOZ source and model weights are not redistributed and remain subject to their respective licenses. [AUTHOR CONFIRM: verify whether redistribution of the derived E3 input is permitted and add the required attribution or remove it from the public package.] The source repository currently requires confirmation of its rights holder and release license before a public availability claim can be finalized.

Ethics statement

The reported experiments used model inference and text derived from an existing dialogue dataset; the repository contains no direct participant recruitment or newly collected human-subject data. [AUTHOR CONFIRM: verify the target institution's determination and replace this statement with the required approval, exemption, or not-applicable wording. Confirm separately whether any human annotators participated outside the repository record.]

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

[AUTHOR CONFIRM: identify every generative-AI or AI-assisted tool used in code, analysis, figures, language editing, or manuscript preparation and state the verified purpose and human review. Suggested venue-compatible form: "During the preparation of this work, the authors used [TOOL/SERVICE] for [PURPOSE]. The authors reviewed and edited the output as needed and take full responsibility for the content of the article."]

Acknowledgements

[AUTHOR CONFIRM: identify any individuals who contributed language editing, technical assistance, or other non-author support, or replace this sentence with "None."]

References

Beigel, K., 2026. stream2sentence: Real-time processing and delivery of sentences from a continuous stream of characters or text chunks. Software release. URL: https://github.com/KoljaB/stream2sentence/releases/tag/v1.0.0. version 1.0.0, released 24 June 2026; accessed 3 September 2026.

Chen, J., Hu, Y., Li, J., Li, K., Liu, K., Li, W., Li, X., Li, Z., Shen, F., Tang, X., Wei, M., Wu, Y., Xie, F., Xu, K., Xie, K., 2025. FireRedChat: A pluggable, full-duplex voice interaction system with cascaded and semi-cascaded implementations. URL: https://arxiv.org/abs/2509.06502, arXiv:2509.06502.

Défossez, A., Mazaré, L., Orsini, M., Royer, A., Pérez, P., Jégou, H., Grave, E., Zeghidour, N., 2024. Moshi: A speech-text foundation model for real-time dialogue. URL: https://arxiv.org/abs/2410.00037, arXiv:2410.00037.

Du, Z., et al., 2024. CosyVoice 2: Scalable streaming speech synthesis with large language models. URL: https://arxiv.org/abs/2412.10117, arXiv:2412.10117.

Edwards, D.W., 2025. Impacts of telecommunications latency on the timing of speaker transitions. Speech Communication 171, 103226. URL: https://doi.org/10.1016/j.specom.2025.103226, doi:10.1016/j.specom.2025.103226.

Eric, M., Goel, R., Paul, S., Sethi, A., Agarwal, S., Gao, S., Kumar, A., Goyal, A.K., Ku, P., Hakkani-Tur, D., 2020. MultiWOZ 2.1: A consolidated multi-domain dialogue dataset with state corrections and state tracking baselines, in: Proceedings of the Twelfth Language Resources and Evaluation Conference, European Language Resources Association. pp. 422–428. URL: https://aclanthology.org/2020.lrec-1.53/.

Hooper, C., Kang, M., Moon, S., Lee, N., Wen, E., Wawrzynek, J., Mahoney, M.W., Shao, Y.S., Gholami, A., Keutzer, K., 2026. Speculative interaction agents: Building real-time agents with asynchronous I/O and speculative tool calling. URL: https://arxiv.org/abs/2605.13360, arXiv:2605.13360.

Hugging Face, 2025a. KV cache strategies. Transformers documentation. URL: https://huggingface.co/docs/transformers/v4.57.1/en/kv_cache. version 4.57.1; accessed 3 September 2026.

Hugging Face, 2025b. Transformers v4.57.1 cache utilities: DynamicCache and crop. Source code. URL: https://github.com/huggingface/transformers/blob/v4.57.1/src/transformers/cache_utils.py. version 4.57.1; accessed 3 September 2026.

Jiang, A.Q., et al., 2023. Mistral 7B. URL: https://arxiv.org/abs/2310.06825, arXiv:2310.06825.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J., Zhang, H., Stoica, I., 2023. Efficient memory management for large language model serving with PagedAttention, in: Proceedings of the 29th Symposium on Operating Systems Principles, ACM. pp. 611–626. URL: https://doi.org/10.1145/3600006.3613165, doi:10.1145/3600006.3613165.

Li, J., Lou, J., Li, J., 2026. IntentKV: Cross-turn intent-aware KV cache pruning for agent inference. URL: https://arxiv.org/abs/2606.09916, arXiv:2606.09916.

LiveKit, n.d. Events and error handling. LiveKit documentation. URL: https://docs.livekit.io/agents/build/events/. accessed 3 September 2026.

Mai, L., 2026. RelayS2S: A dual-path speculative generation for real-time dialogue. URL: https://arxiv.org/abs/2603.23346, arXiv:2603.23346.

Microsoft, 2026. Handle voice interruptions in chat history. Microsoft Learn documentation. URL: https://learn.microsoft.com/en-us/azure/ai-services/speech-service/how-to-voice-live-auto-truncation. published 28 April 2026; accessed 3 September 2026.

Mistral AI, 2024. Mistral-7B-Instruct-v0.3 model card. Hugging Face model card. URL: https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3. accessed 3 September 2026.

Morrone, G., Cornell, S., Serafini, L., Zovato, E., Brutti, A., Squartini, S., 2024. End-to-end integration of speech separation and voice activity detection for low-latency diarization of telephone conversations. Speech Communication 161, 103081. URL: https://doi.org/10.1016/j.specom.2024.103081, doi:10.1016/j.specom.2024.103081.

OpenAI, n.d. Realtime conversations. OpenAI API documentation. URL: https://developers.openai.com/api/docs/guides/realtime-conversations. accessed 3 September 2026.

Radford, A., Kim, J.W., Xu, T., Brockman, G., Mcleavey, C., Sutskever, I., 2023. Robust speech recognition via large-scale weak supervision, in: Proceedings of the 40th International Conference on Machine Learning, PMLR. pp. 28492–28518. URL: https://proceedings.mlr.press/v202/radford23a.html.

Schwarz, A., He, D., Segbroeck, M.V., Hethnawi, M., Rastrow, A., 2023. Personalized predictive ASR for latency reduction in voice assistants, in: INTERSPEECH 2023, ISCA. pp. 745–749. URL: https://doi.org/10.21437/Interspeech.2023-211, doi:10.21437/Interspeech.2023-211.

Yang, A., et al., 2024. Qwen2 technical report. URL: https://arxiv.org/abs/2407.10671, arXiv:2407.10671.

Zheng, J., Cheng, G., Wang, X., Zhao, Q., Yan, Y., 2026. Multilevel contextual prompting for conversational ASR: Unifying conversation history and hotwords with speech LLM. Speech Communication 183, 103458. URL: https://doi.org/10.1016/j.specom.2026.103458, doi:10.1016/j.specom.2026.103458.

Zheng, L., Yin, L., Xie, Z., et al., 2024. SGLang: Efficient execution of structured language model programs, in: Advances in Neural Information Processing Systems, pp. 62557–62583. URL: https://doi.org/10.52202/079017-2000, doi:10.52202/079017-2000.

Zou, W., Miao, Y., Ma, Z., Xu, J., Gao, J., Hao, J., He, R., Xu, J., 2026. LTS-VoiceAgent: A listen-think-speak framework for efficient streaming voice interaction via semantic triggering and incremental reasoning. URL: https://arxiv.org/abs/2601.19952, arXiv:2601.19952.